AI Engineering Implementation Brief

Retail Analytics Assistant — Technical Documentation *[Github]*

Candidate: Alakh Babbar

College: Dayalbagh Educational Institute, Agra

Environment: Single Jupyter notebook, Python, individually implemented.

1. Overview

This document describes a notebook-based, AI-powered Retail Analytics Assistant built for the FMCG analytics assessment. It accepts a natural-language question and answers it using structured data (SQL over SQLite), business knowledge (retrieval over a markdown knowledge base), or both, depending on what the question actually requires. No model was trained or fine-tuned; the implementation is entirely orchestration — prompt engineering, SQL generation, retrieval, and response synthesis over an existing instruction-tuned model.

The dataset backing the system is sales_data.csv, approximately 12,000 retail transaction records. It is loaded into SQLite at notebook start; no manual schema is written.

1.1 Technology Stack

Component	Choice
Language / runtime	Python, Jupyter Notebook
Database	SQLite
Data handling	Pandas, NumPy
Embeddings	Sentence-Transformers (all-MiniLM-L6-v2)
Vector Database	ChromaDB
LLM	Qwen/Qwen2.5-7B-Instruct via Hugging Face Inference API
Config / secrets	python-dotenv
Structured I/O	JSON

2. Architecture

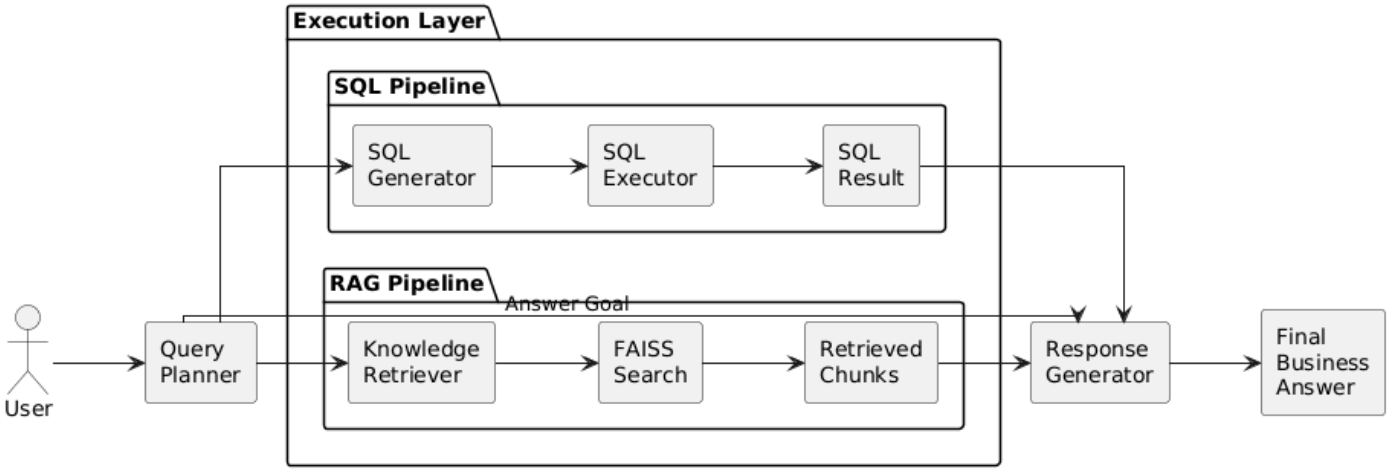
The system is a single orchestrator class, RetailAnalyticsAssistant, composed of four collaborating components rather than one monolithic prompt:

- Planner — reads the question and outputs a small JSON object stating which tools are needed and what each should do: needs_sql, needs_rag, sql_task, rag_task, answer_goal.
- SQL Generator + Executor — turns a SQL task description into a query against the live schema, then runs it.
- Knowledge Retriever — embeds the retrieval task and returns the top matching chunks from knowledge.md.

- Response Generator — combines the original question, the planner's answer goal, the SQL result (if any), and the retrieved knowledge (if any) into a final answer.

Each component has one job and one output contract (usually JSON), which keeps failures localized — if SQL generation fails, that is visible and separable from a retrieval failure or a synthesis failure, rather than being buried inside one large end-to-end prompt.

2.1 Pipeline



If the planner determines that neither SQL nor retrieval is needed — a greeting, an out-of-scope question, or something too vague to act on — the Response Generator still runs, using only the planner's answer_goal, and produces a clarification response. There is no hardcoded fallback string; the same generator handles this case as any other, just with empty evidence inputs.

3. Dataset

sales_data.csv (~12,000 rows) is read with Pandas and written into a SQLite database at notebook startup using DataFrame.to_sql(). No CREATE TABLE statement is authored by hand — the table and column types are inferred from the DataFrame at load time.

Major columns: transaction_id, week_start, region, state, city, store_id, product_id, product_name, category, brand, promotion_name, promotion_type, promotion_active, units_sold, revenue, discount_amount, inventory_before, inventory_after, customer_rating.

3.1 Schema Inspection

Before any SQL is generated, the notebook queries SQLite's own metadata (table list, column names, declared types) and pulls a small number of sample values per column. This inspection result — not a hardcoded schema string — is what gets inserted into the SQL generation prompt. The practical effect: column names are never hand-typed into a prompt anywhere in the codebase, so a change to the CSV's columns does not require touching the SQL generation logic. This inspection runs once at notebook startup, and the same result is reused for every question in the session.

3.2 Why SQLite

SQLite was selected because it is lightweight, requires no external server, integrates directly with Pandas, and is well suited to a notebook-based prototype where the focus is AI orchestration rather than database administration. It gives the assistant a real query engine — proper filtering, joins, and aggregation — without introducing any infrastructure to set up or maintain for what is a single-session, single-user assessment.

4. Knowledge Base and Retrieval

knowledge.md is a separate, hand-authored markdown document holding KPI formulas (Promotion Lift, Baseline Revenue), inventory concepts, aggregation rules, and business terminology. It exists specifically so that business definitions do not have to be re-derived by the LLM at answer time, and so they are not scattered across every prompt as inline instructions.

4.1 Chunking and Embedding

The document is split along its markdown section headers (each metric, rule, or definition is written as a self-contained block, by design, so it retrieves cleanly on its own). Each chunk is embedded once at notebook startup using the `all-MiniLM-L6-v2` Sentence Transformer model and stored in a ChromaDB collection. At query time, ChromaDB computes semantic similarity between the embedded query and stored document embeddings to retrieve the most relevant knowledge chunks.

4.2 Retrieval

At query time, the Knowledge Retriever embeds the planner's `rag_task` and queries the ChromaDB collection to retrieve the most semantically relevant knowledge chunks. The retriever itself performs no language generation; it is purely responsible for semantic retrieval.

5. LLM Configuration

A single model, Qwen/Qwen2.5-7B-Instruct, served through the Hugging Face Inference API, is used for all three generation steps: planning, SQL generation, and final response synthesis. Temperature is set near zero across all three calls. This was a deliberate choice for an analytical assistant: the planner's tool-selection and the SQL generator's query structure should be as repeatable as possible for the same question, and creative variation has no value in either step. All task-specific behavior comes from prompt structure and the schema/knowledge context injected into each call, not from any model training.

An open-source, instruction-tuned model was chosen over a proprietary one specifically because it keeps the implementation reproducible, avoids vendor lock-in, and demonstrates that the solution's reasoning quality comes from the orchestration design rather than from a closed, commercially-licensed API.

6. Query Planner

The first version of this system used a hard intent router that classified each question as SQL, RAG, or Hybrid before doing anything else. That was replaced with a planner for a specific reason: a three-way classification forces every question into exactly one bucket, but a question like "did the campaign improve sales in the South region" genuinely needs both a computed number and a definition to interpret it — it isn't really "Hybrid" as a fourth category so much as SQL and RAG being independently, simultaneously true. The planner expresses that directly as two independent booleans instead of a single label.

6.1 Planner Output Contract

```
{
  "needs_sql": true,
  "needs_rag": true,
  "sql_task": "Compare total revenue for the South region
              during the campaign period vs. the 4 weeks before it",
  "rag_task": "Definition and formula for Promotion Lift",
  "answer_goal": "State whether the campaign improved South region
                  sales, using the lift figure to support the answer"
}
```

The planner is instructed to only plan, never to answer. This separation matters in practice — a planner that starts answering tends to state numbers it hasn't actually computed, which reintroduces the hallucination risk the whole pipeline is designed to avoid.

7. SQL Generator

Given a sql_task string, the injected schema, and the aggregation/business rules from the knowledge base, the SQL Generator produces exactly one query. Its output contract is JSON with two fields:

```
{
  "sql": "SELECT region, SUM(revenue) AS total_revenue ...",
  "summary": "Total revenue by region for the given period"
}
```

The generator does not execute anything itself and does not see the question's answer_goal — only the SQL task. This keeps its responsibility narrow: translate a well-scoped analytical task into correct SQL against a known schema, nothing else.

7.1 SQL Executor

The generated query string is executed directly against the SQLite connection with Pandas' read_sql_query, producing a DataFrame. The DataFrame is then converted to a compact text/markdown table before being passed downstream — the Response Generator never sees a DataFrame object or touches the database connection itself.

8. Response Generator

The final step combines four inputs: the original user question, the planner's answer_goal, the SQL result table (if produced), and the retrieved knowledge chunks (if produced). It is the only component that produces the natural-language answer the user actually sees, and it never queries the database or the vector index directly — it only synthesizes over what it is handed.

Conceptually, its prompt instructs the model to: state the numeric result plainly if one was computed, incorporate the relevant business definition only where it clarifies the numeric result, and avoid inventing figures not present in the SQL table. When neither SQL nor retrieval produced anything (the no-tool-needed case), the same prompt structure is used with both evidence fields empty, and the model is instructed to ask a clarifying question grounded in the answer_goal rather than guessing at intent.

9. Class / Module Responsibilities

Component	Responsibility
RetailAnalyticsAssistant	Top-level orchestrator; owns the planner, generator, executor, retriever, and response generator, and runs the pipeline in order.
QueryPlanner	Produces needs_sql / needs_rag / sql_task / rag_task / answer_goal. Never answers.
SQLGenerator	Turns a sql_task + schema + business rules into one SQL query.
SQLExecutor	Runs SQL against SQLite; returns a DataFrame and a text table.
KnowledgeRetriever	Embeds a rag_task and returns top-k chunks from ChromaDB. No generation.
ResponseGenerator	Synthesizes the final natural-language answer from all gathered evidence.

10. Testing Performed

Testing was manual and question-driven rather than a formal automated test suite — appropriate for a notebook prototype built individually within the assessment's time constraints. The example questions listed in the assessment brief, plus several variations, were run end-to-end and the intermediate outputs at each pipeline stage (planner JSON, generated SQL, retrieved chunks, final answer) were inspected by hand to confirm each stage was doing what it claimed to do, not just that the final answer looked plausible.

Questions specifically tested included revenue-by-region comparisons, promotion-lift questions for the South region campaign, pure definitional questions ("what is Promotion Lift"), and a couple of deliberately vague questions to check the no-tool-needed clarification path. No load testing, no adversarial prompt testing, and no automated regression suite were built.

11. Assumptions

- The dataset schema is stable within a single session — schema inspection happens once at startup, not per query.
- Business terminology used in questions matches, or is close enough in meaning to, terminology defined in knowledge.md for retrieval to find the right chunk.
- A single SQL query is sufficient to answer the sql_task the planner produces. This does not always hold in practice — see Section 12.
- The Hugging Face Inference API endpoint for Qwen2.5-7B-Instruct is reachable and reasonably low-latency during a session; no offline fallback exists.

12. Implementation Limitations

These are stated directly because they reflect the actual state of the notebook, not a hedge:

- The planner produces exactly one sql_task and one rag_task per question. A question that genuinely requires two independent SQL computations, or two unrelated retrievals, is not decomposed — it is handled as a single (and sometimes under-scoped) task.
- The SQL Generator produces one query with no retry. If the generated SQL is invalid, execution fails and there is no self-correction step that reads the error and tries again.
- There is no SQL verification layer. Generated SQL is assumed to be logically correct if it executes without error — a query that runs but answers the wrong question is not caught.
- The planner does not revise its plan after seeing the SQL result. There is no loop back from "this result doesn't look like it answers the question" to a corrected plan.
- Answer quality for definitional and business-concept questions is bounded by the completeness of knowledge.md — anything not written into that document cannot be retrieved.
- The system is scoped to this structured retail dataset and its accompanying knowledge base. It is not a general-purpose question-answering assistant.

13. Future Improvements

Listed here as direction, not as work already done:

- Multi-step planning — allow the planner to emit a list of SQL/retrieval tasks instead of exactly one of each.
- SQL self-correction — feed execution errors back into a repair prompt instead of failing outright.
- A lightweight SQL verification step (e.g. checking the query's referenced columns and aggregation against the business rules before execution).
- Conversation memory across questions in the same session.
- Basic visualization generation for trend-style questions.
- Execution traces and confidence indicators surfaced to the user, not just the final answer.